AI VIET NAM
@aivietnam.edu.vn

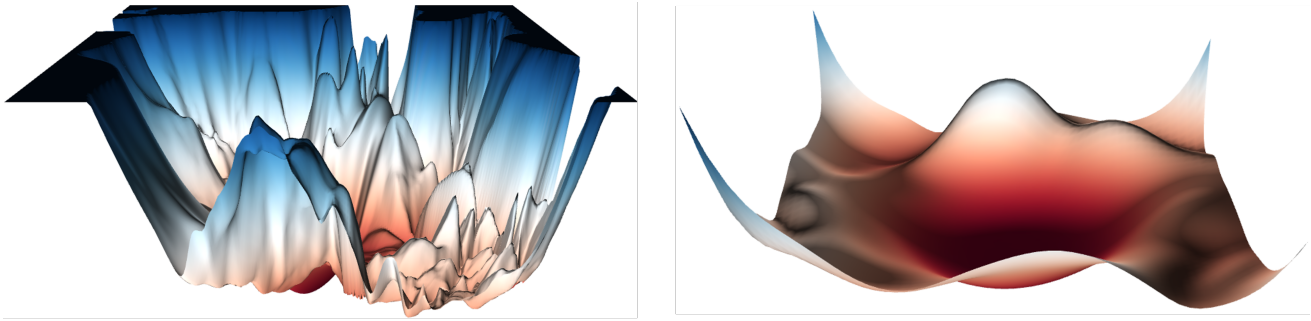
AI VIET NAM – AI COURSE 2025

Tutorial: Optimizer

Nguyễn Thọ Anh Khoa
Nguyễn Phúc Thịnh

I. Giới thiệu

Trong lĩnh vực deep learning, mục tiêu cốt lõi của việc huấn luyện mô hình là tìm ra bộ tham số sao cho loss function đạt giá trị nhỏ nhất. Quá trình này được gọi là "optimization" (tạm dịch: tối ưu hóa). Một cách trực quan, hãy tưởng tượng bạn đang đứng trên một vùng núi hiểm trở trong sương mù và nhiệm vụ của bạn là phải tìm đường xuống thung lũng thấp nhất chỉ bằng cách cảm nhận độ dốc của mặt đất dưới chân mình.



ResNet 56 w/o Shortcut | ResNet 56 with Shortcut

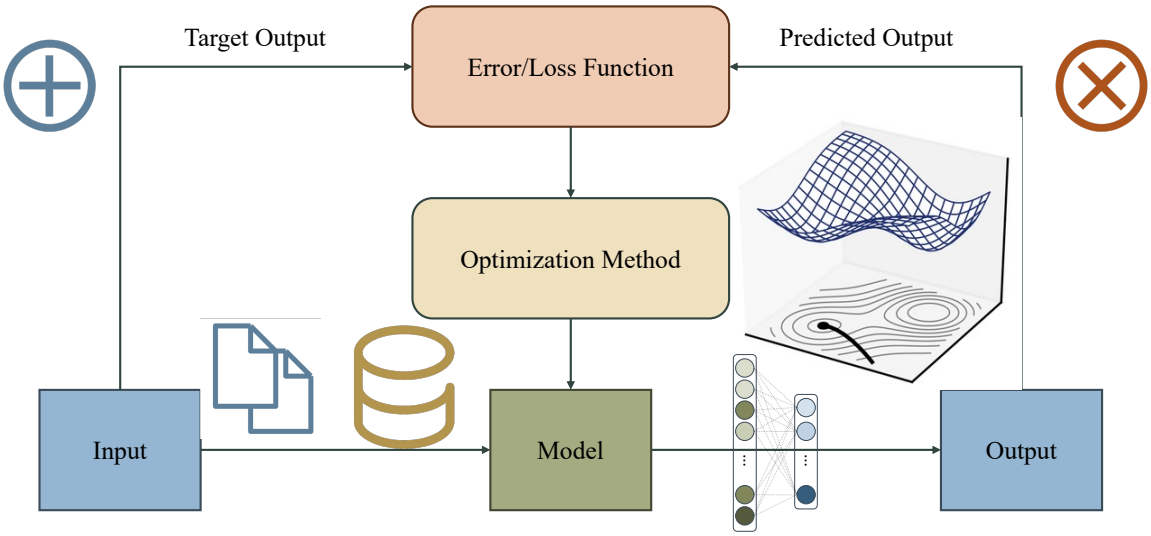
Hình 1: Minh họa bề mặt hàm mất mát phức tạp với nhiều điểm cực trị trong không gian đa chiều. Các thuật toán tối ưu tệ sẽ dễ rơi vào tối ưu cục bộ trong không gian này.

Việc lựa chọn một "optimizer" (tạm dịch: bộ tối ưu hóa) phù hợp đóng vai trò quyết định đến hiệu suất của mạng cần huấn luyện. Một bộ tối ưu hóa tốt không chỉ giúp mô hình hội tụ nhanh hơn mà còn giúp mạng tránh khỏi những vùng mà đạo hàm bị bão hòa. Nếu không có các thuật toán tối ưu hóa thông minh, việc huấn luyện các mô hình lớn với hàng triệu tham số sẽ trở nên bất khả thi do các thách thức sau:

Thách thức	Tác động đến quá trình huấn luyện
"Local minima"	Mô hình bị kẹt tại các thung lũng nông, không đạt được độ chính xác tốt nhất.
"Saddle points"	Các điểm yên ngựa khiến độ dốc bằng không, làm quá trình học bị đình trệ.
"High curvature"	Các vùng có độ dốc thay đổi đột ngột gây ra sự dao động mạnh và mất ổn định.

Bảng 1: Các rào cản phổ biến trên bề mặt hàm mất mát mà bộ tối ưu hóa cần vượt qua.

Về bản chất, bộ tối ưu hóa là một thuật toán toán học đóng vai trò điều phối việc cập nhật các tham số của mô hình dựa trên các tín hiệu phản hồi từ hàm mất mát. Thay vì tìm kiếm các tham số tốt nhất một cách ngẫu nhiên, bộ tối ưu hóa sử dụng thông tin từ đạo hàm để đưa ra các quyết định có tính toán về hướng đi và độ lớn của mỗi bước cập nhật.



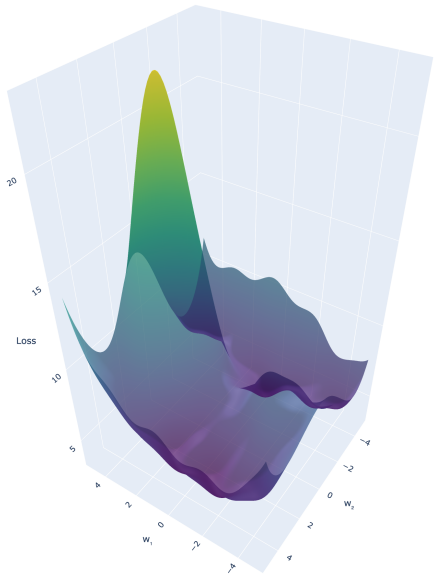
Hình 2: Sơ đồ quy trình cập nhật tham số trong một bước huấn luyện mạng nơ-ron.

Việc hiểu rõ cơ chế này giúp chúng ta nhận ra rằng "optimizer" không chỉ đơn thuần là công thức cộng trừ trọng số, mà là bộ não điều khiển tốc độ và sự ổn định của toàn bộ quá trình học tập.

Trong bài học này, chúng ta sẽ đi từ những khái niệm cơ bản nhất của Gradient Descent để hiểu cách mô hình "cảm nhận" độ dốc, sau đó tiến tới các thuật toán hiện đại hơn như Momentum, RMSProp hay Adam – những công cụ đã trở thành tiêu chuẩn công nghiệp giúp mô hình vượt qua các địa hình phức tạp để đạt tới trạng thái tối ưu.

II. Lý thuyết

Tiếp nối phần dẫn nhập, chúng ta sẽ đi sâu vào chi tiết cấu tạo và nguyên lý hoạt động của các bộ tối ưu hóa phổ biến nhất hiện nay.



Hình 3: Cấu trúc phức tạp của Loss Landscape dùng trong demo.

Để có thể minh hoạ được cách các thuật toán tối ưu hoạt động, chúng ta sẽ sử dụng một không gian hàm loss với 3 đặc điểm phức tạp được nêu ở phần giới thiệu. Trong không gian này có 4 vùng cực tiểu lân cận mà các phương pháp tối ưu cần phải vượt qua.

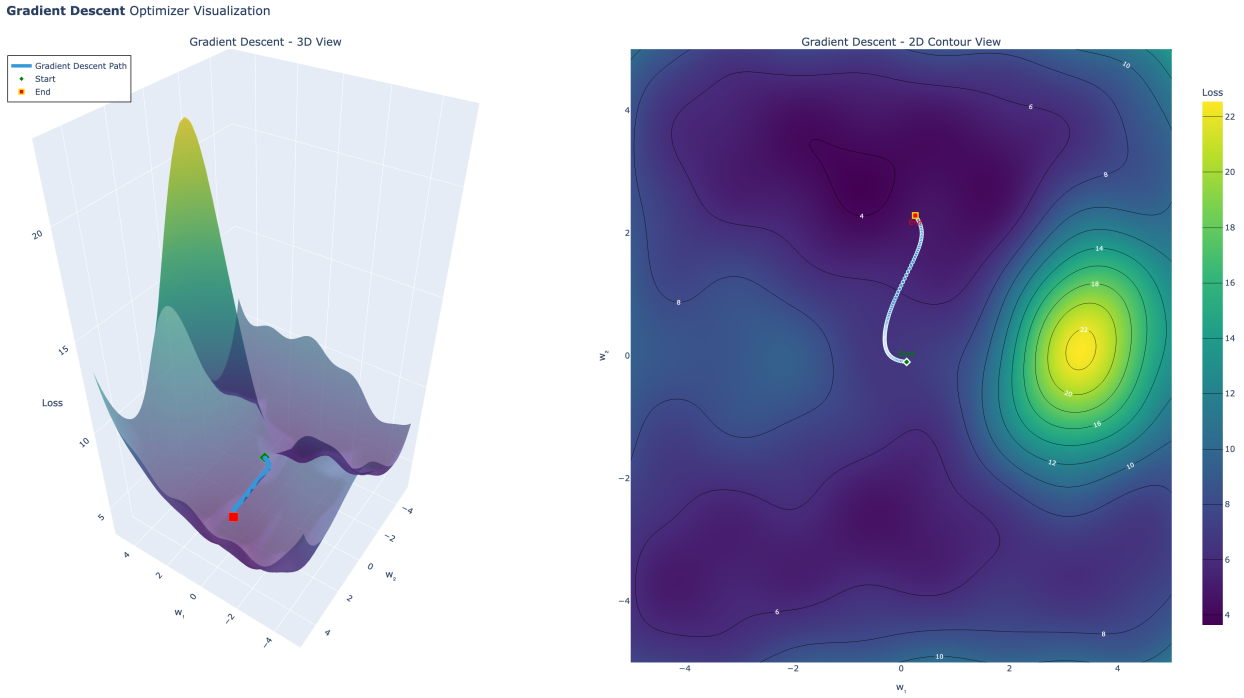
Cho việc thực nghiệm được công bằng hơn, cả 4 phương pháp đều sẽ bắt đầu chung một điểm trên không gian 3 chiều này và có chung số lượng epoch để hội tụ. Vị trí được chọn bắt đầu là điểm nằm trên vùng yên ngựa ở khu vực $(0.5, 0.5)$ của trục toạ độ. Ở đây nằm gần cùng lúc cả 4 cực tiểu toàn cục phức tạp được nói ở trên và gần với một cực tiểu toàn cục mà chúng ta cần đạt được.

Trước khi có thể bắt đầu tìm toạ độ loss bé nhất, thuật toán chúng ta sẽ cần vượt qua vùng bề mặt nằm ngang với $\sigma^2 \approx 0.001$.

II.1. Thuật toán Gradient Descent

Đây là thuật toán nền tảng nhất để tìm điểm "minimum" của một hàm số dựa trên việc tính toán đạo hàm. Để thuật toán hoạt động hiệu quả, hàm mục tiêu cần đảm bảo tính liên tục và có thể lấy đạo hàm tại hầu hết các điểm.

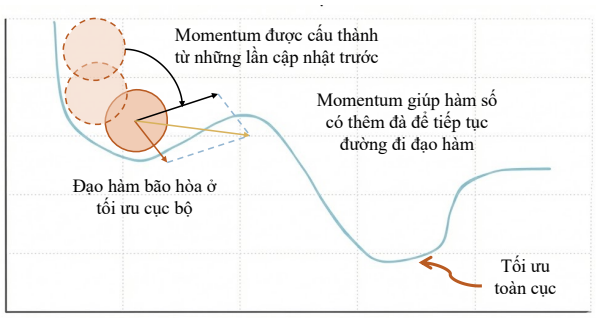
Trong phương pháp này, ta bắt đầu tại một vị trí ngẫu nhiên trên bề mặt hàm số. Tại mỗi bước, thuật toán tính toán "gradient" (tạm dịch: độ dốc) và di chuyển ngược hướng với độ dốc đó để tiến dần về vùng có giá trị thấp hơn. Quá trình này được thực hiện lặp đi lặp lại cho đến khi mô hình hội tụ hoặc đạt được điều kiện dừng xác định trước.



Hình 4: Quá trình di chuyển từng bước của Gradient Descent từ điểm khởi tạo đến điểm cực tiểu cục bộ.

II.2. Gradient descent kết hợp Momentum

Phương pháp này được phát triển để khắc phục nhược điểm của Gradient Descent truyền thống khi đối mặt với các "local minimum" (tạm dịch: cực tiểu địa phương). Gradient Descent cơ bản thường dễ bị mắc kẹt tại những thung lũng nông và không thể tiếp tục tìm kiếm các vùng tối ưu tốt hơn.



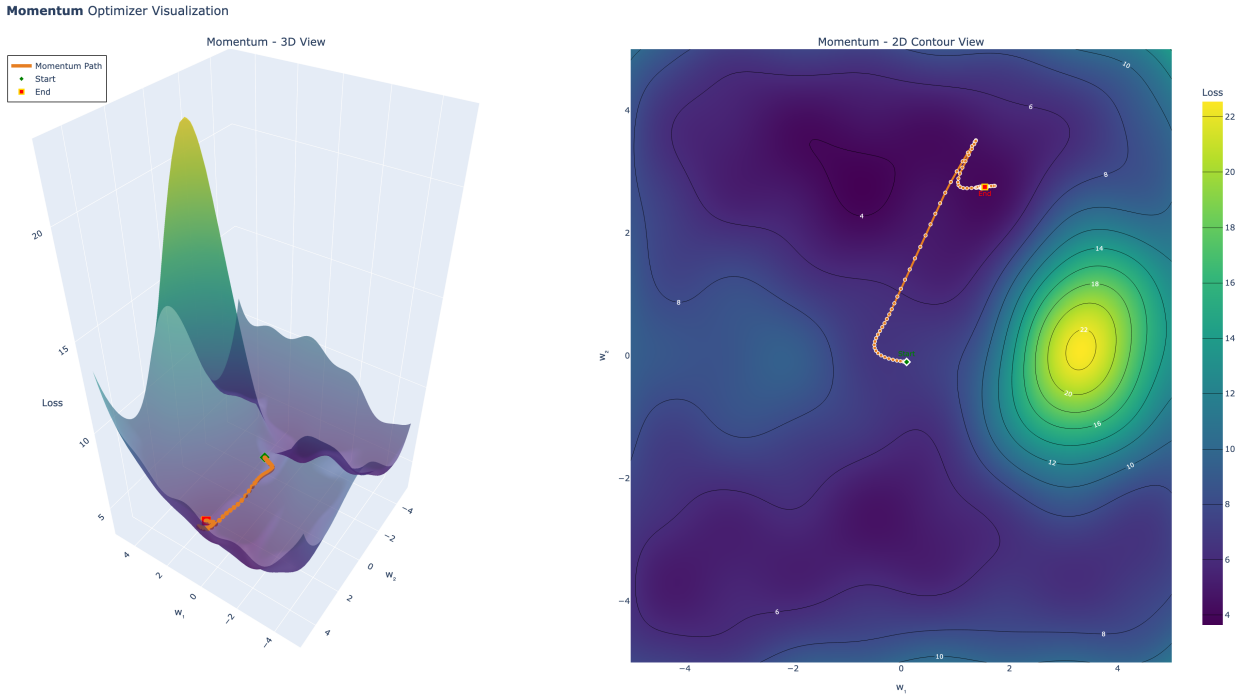
Hình 5: Sự khác biệt giữa việc bị kẹt tại cực tiểu địa phương và việc vượt qua dốc nhờ vào cơ chế đà.

Thuật toán "Momentum" sử dụng cơ chế "exponentially weighted average" để tích lũy các giá trị đạo hàm từ các bước trước đó. Việc kết hợp vận tốc tích lũy này với đạo hàm hiện tại giúp tạo ra một lực đẩy đủ lớn, tương tự như một viên bi đang lăn có đà để vượt qua các gờ dốc nhỏ.

Công thức cập nhật:

$$V_t = \underbrace{\beta V_{t-1}}_{\text{Quán tính}} + \underbrace{(1-\beta) dW_t}_{\text{Lực đẩy mới}}$$

$$W_t = \underbrace{W_{t-1}}_{\text{Vị trí cũ}} - \underbrace{\alpha V_t}_{\text{Bước nhảy}}$$



Hình 6: Quá trình di chuyển từng bước của Momentum từ điểm khởi tạo đến điểm cực tiểu cục bộ.

II.3. Thuật toán RMSProp

Được mở rộng từ ý tưởng của "Adagrad", thuật toán "RMSProp" giới thiệu khả năng "adaptive learning rate" (tạm dịch: tốc độ học thích nghi). Thay vì sử dụng một hệ số học cố định cho tất cả các chiều của tham số, "RMSProp" điều chỉnh bước đi dựa trên độ lớn của các đạo hàm gần nhất.

Ý tưởng cốt lõi là chia đạo hàm hiện tại cho căn bậc hai của trung bình bình phương các đạo hàm trước đó. Điều này giúp làm giảm tốc độ cập nhật ở những hướng có dao động quá lớn và tăng tốc ở những hướng có độ dốc nhỏ, giúp quá trình huấn luyện trở nên ổn định hơn. Chúng ta có công thức cập nhật như sau:

- Tích lũy bình phương đạo hàm (S_t):

$$S_t = \underbrace{\gamma S_{t-1}}_{\text{Ghi nhớ cũ}} + \underbrace{(1-\gamma) dW_t^2}_{\text{Biến động mới}}$$

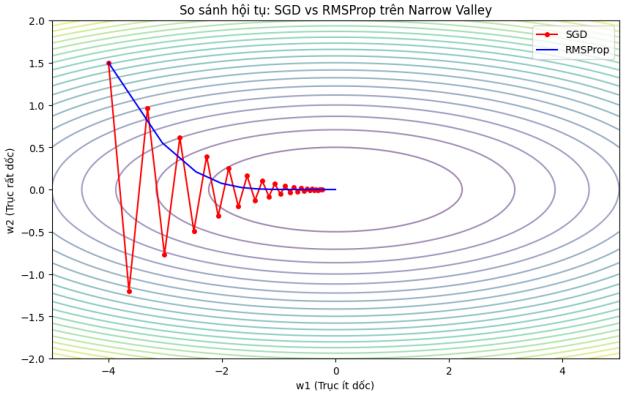
- Cập nhật tham số với tốc độ học thích nghi:

$$W_t = W_{t-1} - \frac{\alpha}{\underbrace{\sqrt{S_t + \epsilon}}_{\text{Bộ điều tiết}}} dW_t$$

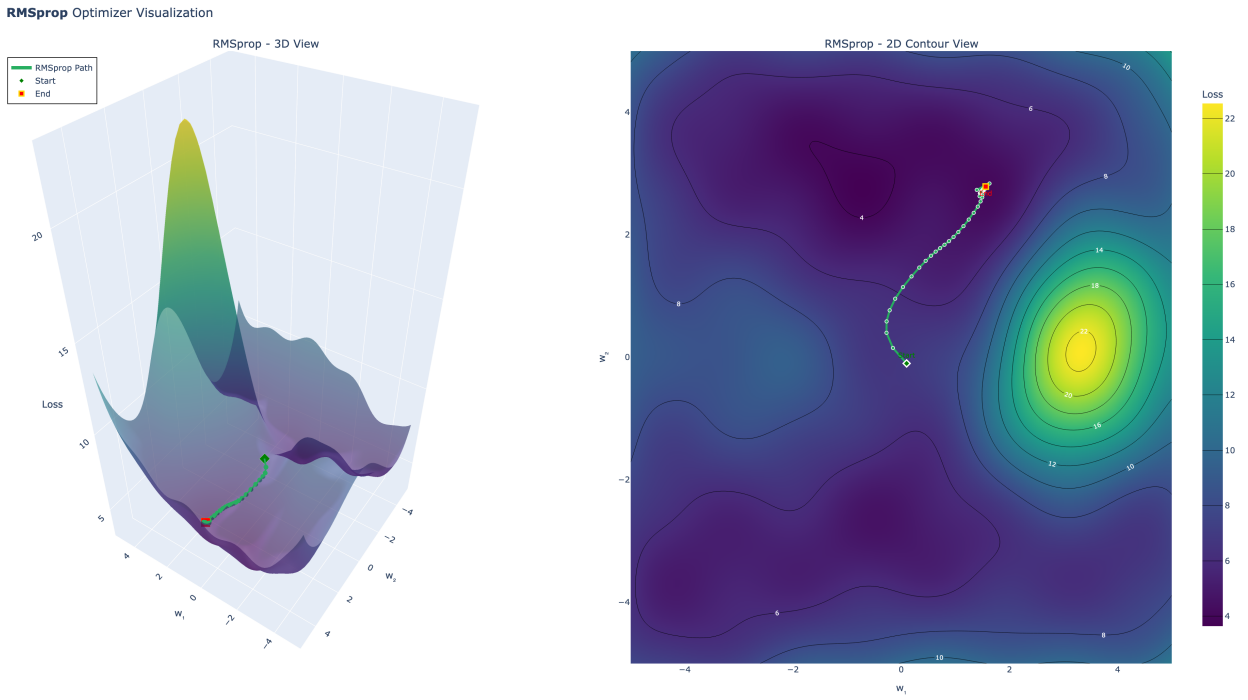
Hình bên phải minh họa quá trình tối ưu trên một hàm Loss dạng narrow valley, nơi gradient theo hướng w_2 (trục tung) lớn hơn nhiều so với hướng w_1 (trục hoành).

SGD: Với tốc độ học cố định, bước nhảy quá lớn so với độ dốc của vách thung lũng, khiến thuật toán liên tục dao động qua lại hai bên vách mà tiến triển rất chậm theo chiều dọc thung lũng. Chúng ta có thể tưởng tượng thuật toán như một chiếc xe đang chuyển hướng nhưng không giảm ga.

RMSProp: Cơ chế tốc độ học thích nghi giúp chia nhỏ bước nhảy ở hướng dốc đứng (w_2) để triệt tiêu dao động, đồng thời phóng to bước nhảy ở hướng thoai thoải (w_1). Kết quả là đường hội tụ mượt mà và trực diện về điểm tối ưu.



Hình 7: So sánh đường đi hội tụ giữa SGD (dao động mạnh) và RMSProp (ổn định) trên địa hình thung lũng hẹp.



Hình 8: Quá trình di chuyển từng bước của RMSProp từ điểm khởi tạo đến điểm cực tiểu cục bộ. Đường đi được tạo ra mượn mà.

II.4. Thuật toán Adam

Thuật toán "Adam" (tên đầy đủ là Adaptive moment estimation) hiện là lựa chọn phổ biến nhất trong thực tế nhờ kết hợp ưu điểm của cả hai phương pháp Momentum và RMSProp. Thuật toán này đồng thời duy trì việc tích lũy đà của đạo hàm và trung bình bình phương đạo hàm.

Cấu trúc của Adam có thể được chia thành ba giai đoạn chính: tích lũy mô-men (Momentum), theo dõi biến động đạo hàm (RMSProp) và hiệu chỉnh sai số (Bias Correction).

$$\underbrace{V_t}_{\text{Mô-men bậc 1}} = \underbrace{\beta_1 V_{t-1}}_{\text{Quán tính cũ}} + \underbrace{(1-\beta_1)dW_t}_{\text{Đạo hàm hiện tại}} \qquad \underbrace{S_t}_{\text{Mô-men bậc 2}} = \underbrace{\beta_2 S_{t-1}}_{\text{Biến động cũ}} + \underbrace{(1-\beta_2)dW_t^2}_{\text{Biến động hiện tại}}$$

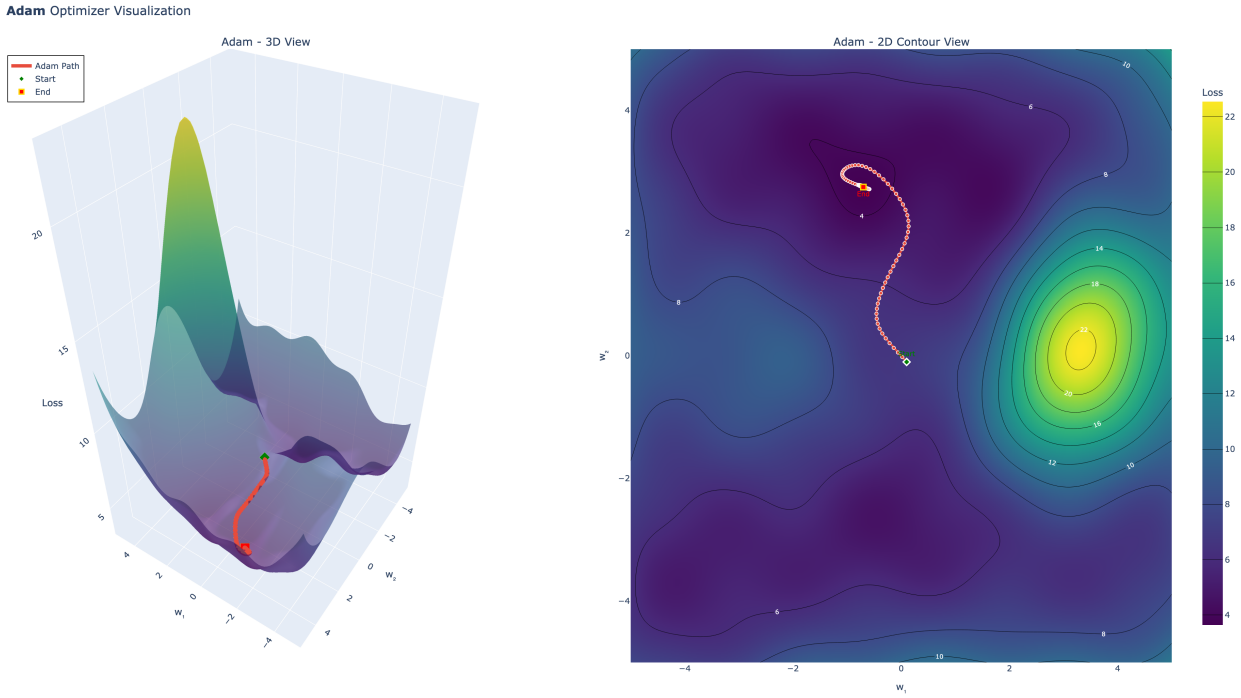
$$\underbrace{\hat{V}_t = \frac{V_t}{1-\beta_1^t}, \quad \hat{S}_t = \frac{S_t}{1-\beta_2^t}}_{\text{Hiệu chỉnh sai số (Bias Correction)}} \qquad W_t = W_{t-1} - \underbrace{\alpha}_{\text{LR}} \cdot \frac{\hat{V}_t}{\sqrt{\hat{S}_t} + \epsilon}$$

Ý nghĩa các thành phần:

- V_t **(First Moment):** Đóng vai trò như một lực quán tính, giúp mô hình giữ được hướng di chuyển ổn định và vượt qua các cực tiểu cục bộ.
- S_t **(Second Moment):** Theo dõi độ lớn của các đạo hàm gần nhất. Nếu một chiều nào đó có đạo hàm quá lớn (dao động mạnh), S_t sẽ tăng lên để kìm hãm bước nhảy lại, giúp ổn định quá trình tối ưu.
- $\hat{V}_t, \hat{S}_t$**:** Giúp khắc phục nhược điểm của việc khởi tạo các giá trị V, S bằng 0, khiến mô hình không bị đình trệ trong những epoch đầu tiên.

Trong quá trình cấu hình bộ tối ưu Adam, việc tinh chỉnh các siêu tham số đóng vai trò quyết định đến hiệu suất hội tụ của mô hình. Ta có hai hệ số điều tiết là β_1 và β_2 thường được giữ cố định lần lượt ở mức 0.9 và 0.999; trong đó β_1 quyết định mức độ ảnh hưởng của các bước đi trong quá khứ (quán tính), còn β_2 kiểm soát cách thức tích lũy trung bình bình phương đạo hàm. Cuối cùng, hệ số ổn định ϵ thường được thiết lập giá trị cực nhỏ khoảng 10^{-8} với mục đích duy nhất là đảm bảo tính ổn định về mặt tính toán, tránh lỗi chia cho không khi đạo hàm tiệm cận về mức triệt tiêu.

Sự kết hợp này giúp "Adam" vừa có khả năng vượt qua các bẫy cực tiểu địa phương, vừa có tốc độ hội tụ rất nhanh nhờ khả năng tự thích nghi hệ số học. Điều này giúp giảm thiểu đáng kể thời gian điều chỉnh tham số thủ công cho người huấn luyện mô hình.



Hình 9: Lộ trình hội tụ mượt mà và nhanh chóng của thuật toán adam trên bề mặt hàm mất mát phức tạp. Đây là bộ tối ưu duy nhất đạt được cực tiểu toàn cục trên bài toán này.

III. Bài Tập và Gợi Ý

III.1. Gợi Ý Các Bước Làm Bài

Bài tập này tập trung vào việc tính toán thủ công các bước trong thuật toán tối ưu hóa và hiện thực hóa các thuật toán này bằng thư viện NumPy. Ngoài ra, phần bài tập nâng cao (optional) sẽ thực hiện thay đổi các thuật toán trên PyTorch để quan sát cách chúng khắc phục hiện tượng triệt tiêu đạo hàm (vanishing gradient).

Các bài tập từ 1 đến 4 sẽ thực hiện tối ưu hóa cho hàm số 2 biến sau:

$$f(w_1, w_2) = 0.1w_1^2 + 2w_2^2$$

Bài 1: Gradient Descent

Công thức cập nhật:

$$W = W - \alpha \cdot dW \tag{1.1}$$

a) **Tính toán thủ công:** Trình bày chi tiết từng bước tìm điểm minimum sau 2 epoch.

- **Epoch 1:**
  - **STEP 1:** Tìm giá trị dw_1 và dw_2 (đạo hàm riêng của hàm đề bài tại điểm khởi tạo $[w_1, w_2]$.
  - **STEP 2:** Dùng công thức (1.1) để cập nhật w_1 và w_2.
- **Epoch 2:**
  - **STEP 3:** Thực hiện tương tự Epoch 1 với giá trị đã cập nhật.

b) **Hiện thực hóa bằng NumPy (30 epoch):**

- **STEP 1:** Xây dựng hàm `df_w` tính đạo hàm riêng và trả về mảng $[dw_1, dw_2]$.
- **STEP 2:** Xây dựng hàm `sgd` thực hiện cập nhật theo công thức (1.1).
- **STEP 3:** Xây dựng hàm `train_p1` để chạy vòng lặp tối ưu hóa.

```python
def df_w(W):
    """
    Thực hiện tính gradient của dw1 và dw2
    Arguments:
    W -- np.array [w1, w2]
    Returns:
    dW -- np.array [dw1, dw2], array chứa giá trị đạo hàm theo w1 và w2
    """

    ################### YOUR CODE HERE ###################

    dW =
    ######################################################

```

```
    return dW
```

```
def sgd(W, dW, lr):
    """
    Thực hiện thuật toán Gradient Descent để update w1 và w2
    Arguments:
    W -- np.array: [w1, w2]
    dW -- np.array: [dw1, dw2], array chứa giá trị đạo hàm theo w1 và w2
    lr -- float: learning rate
    Returns:
    W -- np.array: [w1, w2] w1 và w2 sau khi đã update
    """

    ################### YOUR CODE HERE ###################

    W =
    ######################################################

    return W
```

```
def train_pl(optimizer, lr, epochs):
    """
    Thực hiện tìm điểm minimum của function (1) dựa vào thuật toán
    được truyền vào từ optimizer
    Arguments:
    optimize : function thực hiện thuật toán optimization cụ thể
    lr -- float: learning rate
    epoch -- int: số lượng lần (epoch) lặp để tìm điểm minimum
    Returns:
    results -- list: list các cặp điểm [w1, w2] sau mỗi epoch (mỗi lần cập nhật)
    """

    # initial point
    W = np.array([-5, -2], dtype=np.float32)
    # list of results
    results = [W]
    ################### YOUR CODE HERE ###################
    # Tạo vòng lặp theo số lần epochs
    # tìm gradient dW gồm dw1 và dw2
    # dùng thuật toán optimization cập nhật w1 và w2
    # append cặp [w1, w2] vào list results
    ######################################################

    return results
```

Bài 2: Gradient Descent + Momentum

Công thức cập nhật:

$$V_t = \beta V_{t-1} + (1-\beta) dW_t$$
$$W_t = W_{t-1} - \alpha \cdot V_t \tag{2.1, 2.2}$$

a) **Tính toán thủ công:** Trình bày chi tiết sau 2 epoch (khởi tạo $v_1 = 0, v_2 = 0$).

- **STEP 1:** Tìm dw_1, dw_2 tại điểm hiện tại.
- **STEP 2:** Tính toán vận tốc v_1, v_2 theo công thức (2.1).
- **STEP 3:** Cập nhật tham số theo công thức (2.2).

b) **Hiện thực hóa bằng NumPy:**

- **STEP 1:** Sử dụng lại hàm `df_w`.
- **STEP 2:** Xây dựng hàm `sgd_momentum` cập nhật đồng thời V và W.
- **STEP 3:** Hàm `train_p1` cần khởi tạo thêm biến vận tốc V.

```python
def sgd_momentum(W, dW, lr, V, beta):
    """
    Thực hiện thuật tóan Gradient Descent + Momentum để update w1 và w2
    Arguments:
    W -- np.array: [w1, w2]
    dW -- np.array: [dw1, dw2], array chứa giá trị đạo hàm theo w1 và w2
    lr -- float: learning rate
    V -- np.array: [v1, v2] Exponentially weighted averages gradients
    beta -- float: hệ số long-range average
    Returns:
    W -- np.array: [w1, w2] w1 và w2 sau khi đã update
    V -- np.array: [v1, v2] Exponentially weighted averages gradients sau khi đã c
                                            ập nhật
    """
    ##################### YOUR CODE HERE ####################

    V =
    W =
    ##########################################################
    return W, V
```

```python
def train_p1(optimizer, lr, epochs):
    # initial
    W = np.array([-5, -2], dtype=np.float32)
    V = np.array([0, 0], dtype=np.float32)
    results = [W]
    ##################### YOUR CODE HERE ####################
    # Tạo vòng lặp theo số lần epochs
    # tìm gradient dW gồm dw1 và dw2
    # dùng thuật toán optimization cập nhật w1, w2, v1, v2
    # append cặp [w1, w2] vào list results
```

```
    ###########################################################
    return results
```

Bài 3: RMSProp

Công thức cập nhật:

$$S_t = \gamma S_{t-1} + (1-\gamma) dW_t^2$$
$$W_t = W_{t-1} - \alpha \cdot \frac{dW_t}{\sqrt{S_t + \epsilon}} \quad (3.1,\ 3.2)$$

a) **Tính toán thủ công:** Trình bày chi tiết từng bước thực hiện tìm điểm minimum theo thuật toán RMSProp (tìm w_1 và w_2 sau 2 epoch) với epoch = 2.

- **Epoch 1:**
  - **STEP 1:** Tìm giá trị dw_1 và dw_2 là giá trị đạo hàm riêng của hàm đề bài theo w_1 và w_2 tại điểm khởi tạo $[w_1, w_2]$.
  - **STEP 2:** Tìm giá trị tích lũy bình phương đạo hàm s_1 và s_2 dựa vào dw_1, dw_2 vừa tìm được ở Step 1 theo công thức (3.1).
  - **STEP 3:** Dùng công thức (3.2) để cập nhật giá trị mới cho w_1 và w_2. Hoàn thành epoch = 1.
- **Epoch 2:**
  - **STEP 4:** Thực hiện tương tự các bước STEP 1, STEP 2 và STEP 3 như trên với các giá trị w_1, w_2 đã được cập nhật từ epoch = 1.

b) **Hiện thực hóa bằng NumPy:**

- **STEP 1:** Xây dựng hàm `rmsprop` theo cơ chế điều chỉnh tốc độ học thích nghi.
- **STEP 2:** Hàm huấn luyện khởi tạo biến tích lũy bình phương đạo hàm S.

```
def RMSProp(W, dW, lr, S, gamma):
    """
    Thực hiện thuật tóan RMSProp để update w1 và w2
    Arguments:
    W -- np.array: [w1, w2]
    dW -- np.array: [dw1, dw2], array chứa giá trị đạo hàm theo w1 và w2
    lr -- float: learning rate
    S -- np.array: [s1, s2] Exponentially weighted averages bình phương gradients
    gamma -- float: hệ số long-range average
    Returns:
    W -- np.array: [w1, w2] w1 và w2 sau khi đã update
    S -- np.array: [s1, s2] Exponentially weighted averages bình phương gradients
                                        sau khi đã cập nhật
    """
```

```
    epsilon = 1e-6
    ##################### YOUR CODE HERE ####################

    S =

    W =
    ##########################################################
    return W, S
```

```
def train_p1(optimizer, lr, epochs):
    # initial
    W = np.array([-5, -2], dtype=np.float32)
    V = np.array([0, 0], dtype=np.float32)
    S = np.array([0, 0], dtype=np.float32)
    results = [W]
    ##################### YOUR CODE HERE ####################
    # Tạo vòng lặp theo số lần epochs
    # tìm gradient dW gồm dw1 và dw2
    # dùng thuật toán optimization cập nhật w1, w2, s1, s2, v1, v2
    # append cặp [w1, w2] vào list results
    # các bạn lưu ý mỗi lần lặp nhớ lấy t (lần thứ t lặp) và t bắt đầu bằng 1


    ##########################################################
    return results
```

Bài 4: Adam

Công thức cập nhật:

$$
\begin{aligned}
V_t &= \beta_1 V_{t-1} + (1-\beta_1) dW_t \\
S_t &= \beta_2 S_{t-1} + (1-\beta_2) dW_t^2 \\
\hat{V}_t &= \frac{V_t}{1-\beta_1^t}, \quad \hat{S}_t = \frac{S_t}{1-\beta_2^t} \\
W_t &= W_{t-1} - \alpha \cdot \frac{\hat{V}_t}{\sqrt{\hat{S}_t} + \epsilon}
\end{aligned}
\tag{4.1 – 4.5}
$$

a) **Tính toán thủ công:** Yêu cầu trình bày chi tiết từng bước thực hiện tìm điểm minimum theo thuật toán Adam (tìm w_1 và w_2 sau 2 epoch) với epoch = 2.

- **Epoch 1:**
  - **STEP 1:** Tìm giá trị dw_1 và dw_2 là giá trị đạo hàm riêng của hàm đề bài theo w_1 và w_2 tại điểm khởi tạo $[w_1, w_2]$.
  - **STEP 2:** Tìm giá trị v_1 và v_2 dựa vào dw_1 và dw_2 vừa tìm được ở Step 1 theo công thức (4.1).

  - **STEP 3:** Tìm giá trị s_1 và s_2 dựa vào dw_1 và dw_2 vừa tìm được ở Step 1 theo công thức (4.2).
  - **STEP 4:** Thực hiện hiệu chỉnh sai số (bias-correction) cho V và S để thu được $\hat{V}$ (V_corr) và $\hat{S}$ (S_corr) theo công thức (4.3) và (4.4). Kết quả thu được là $v_{corr1}, v_{corr2}, s_{corr1}$ và s_{corr2}.
  - **STEP 5:** Dùng công thức (4.5) để cập nhật giá trị mới cho w_1 và w_2. Hoàn thành epoch = 1.
- **Epoch 2:**
  - **STEP 6:** Thực hiện tương tự các bước từ STEP 1 đến STEP 5 như trên với các giá trị w_1 và w_2 đã được cập nhật từ epoch = 1.

b) **Hiện thực hóa bằng NumPy:**

- **STEP 1:** Xây dựng hàm `adam` kết hợp cả Momentum và RMSProp kèm hiệu chỉnh.
- **STEP 2:** Hàm huấn luyện quản lý cả hai biến trạng thái V và S.

```python
def Adam(W, dW, lr, V, S, beta1, beta2, t):
    """
    Thực hiện thuật tóan Adam để update w1 và w2
    Arguments:
    W -- np.array: [w1, w2]
    dW -- np.array: [dw1, dw2], array chứa giá trị đạo hàm theo w1 và w2
    lr -- float: learning rate
    V -- np.array: [v1, v2] Exponentially weighted averages gradients
    S -- np.array: [s1, s2] Exponentially weighted averages bình phương gradients
    beta1 -- float: hệ số long-range average cho V
    beta2 -- float: hệ số long-range average cho S
    t -- int: lần thứ t update (bắt đầu bằng 1)
    Returns:
    W -- np.array: [w1, w2] w1 và w2 sau khi đã update
    V -- np.array: [v1, v2] Exponentially weighted averages gradients sau khi đã cập nhật
    S -- np.array: [s1, s2] Exponentially weighted averages bình phương gradients sau khi đã cập nhật
    """
    epsilon = 1e-6
    ##################### YOUR CODE HERE ####################
    V =
    S =

    W =
    ##########################################################
    return W, V, S
```

Bài 5: Vấn đề Vanishing Gradient (Optional)

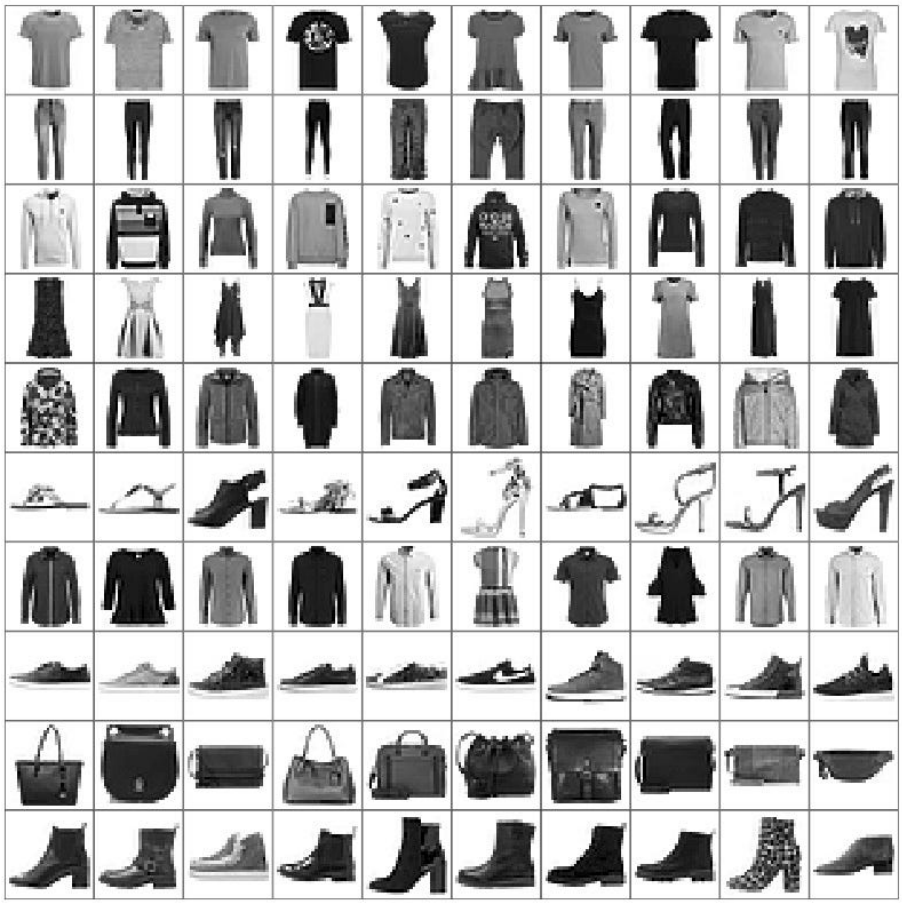
Trong bài tập này, bạn sẽ làm việc với một file mã nguồn để huấn luyện mô hình MLP (Multi-Layer Perceptron) phân loại 10 lớp đối tượng thời trang trên tập dữ liệu FashionMNIST.

Mục tiêu: Mô hình được thiết kế có chủ đích để xảy ra hiện tượng *Vanishing Gradient* (triệt tiêu đạo hàm). Nhiệm vụ của bạn là thay thế các Optimizer đã thực hiện ở các bài trước (SGD, Momentum, RMSProp, Adam) vào mô hình này, sau đó quan sát và đánh giá hiệu năng của từng thuật toán trong việc khắc phục vấn đề Vanishing Gradient.

Giới thiệu về tập dữ liệu FashionMNIST

FashionMNIST là tập dữ liệu hình ảnh các sản phẩm của Zalando, được giới thiệu như một phương án thay thế thách thức hơn cho tập MNIST truyền thống.

- **Đặc điểm:** Mỗi mẫu là một hình ảnh xám kích thước 28×28 pixel.
- **Phân loại:** 10 lớp (Áo sơ mi, T-shirt, Túi xách, Giày, ...).
- **Quy mô:** 60,000 hình ảnh cho tập huấn luyện (train) và 10,000 hình ảnh cho tập kiểm thử (test).



Hình 10: Vài mẫu dữ liệu FashionMNIST.

1. Chuẩn bị dữ liệu

Cấu hình tham số (Cell 1):

- `batch_size = 512`: Chỉ định số lượng mẫu dữ liệu đưa qua mạng trong mỗi lần cập nhật trọng số.
- `num_epochs = 300`: Số lần lặp lại toàn bộ tập dữ liệu.
- `lr = 0.01`: Tốc độ học (learning rate), tham số quyết định bước nhảy của mô hình dựa trên gradient của hàm mất mát.

Khởi tạo DataLoader (Cell 2):

- `train_dataset`: Tải và biến đổi tập FashionMNIST thành dạng Tensor. Dữ liệu được lưu trữ tại thư mục `'./data'`.
- `train_loader`: Bộ tải dữ liệu hỗ trợ xáo trộn (shuffle) dữ liệu huấn luyện.
- `test_dataset` & `test_loader`: Tương tự tập train nhưng thiết lập `train=False` để phục vụ việc đánh giá mô hình.

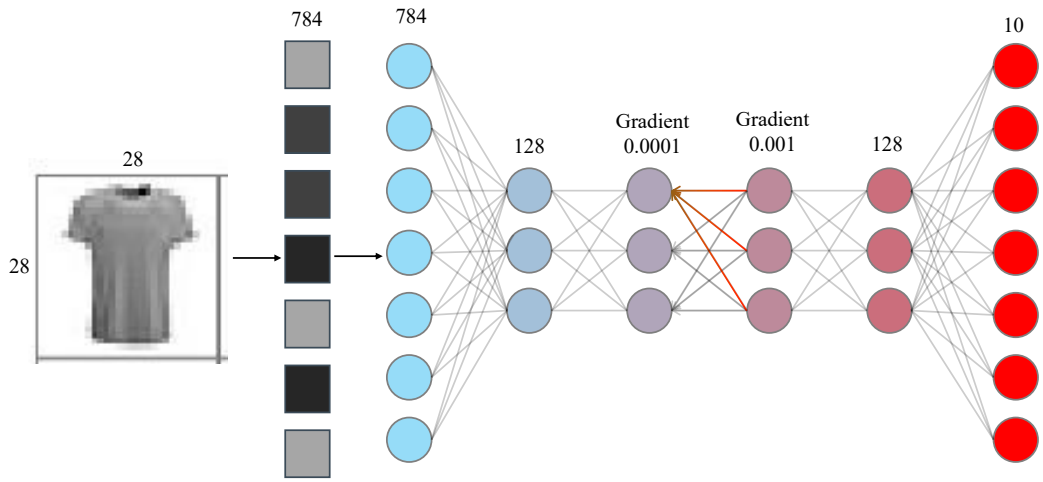
```
batch_size = 512
num_epochs = 300
lr = 0.01

train_dataset = FashionMNIST(root='./data', train=True, download=True, transform=
                                        transforms.ToTensor())
train_loader = DataLoader(train_dataset, batch_size, shuffle=True)
test_dataset = FashionMNIST(root='./data', train=False, download=True, transform=
                                        transforms.ToTensor())
test_loader = DataLoader(test_dataset, batch_size)
```

2. Xây dựng và Thiết lập mô hình

Định nghĩa kiến trúc mạng (Cell 1):

- Sử dụng lớp `MLP` kế thừa từ `nn.Module` của PyTorch.



Hình 11: Cấu trúc mạng được sử dụng trong bài.

- **Kiến trúc:**
  - `layer1`: Input (784) → Hidden (128).
  - `layer2, 3, 4, 5`: Các lớp ẩn trung gian (Hidden → Hidden).
  - `output`: Hidden (128) → Output (10).
- **Hàm kích hoạt:** Sử dụng **Sigmoid** sau mỗi lớp ẩn. Đây là nguyên nhân chính gây ra hiện tượng Vanishing Gradient khi mạng trở nên sâu.
- **Forward pass:** Dữ liệu được làm phẳng (*Flatten*) trước khi đi qua các lớp Fully Connected.

Khởi tạo các thành phần (Cell 2):

- **Model:** Khởi tạo với số chiều 784 → 128 → 10.
- **Loss function:** Sử dụng `CrossEntropyLoss` cho bài toán phân loại đa lớp.
- **Optimizer:** Mặc định khởi tạo với `SGD`. *(Lưu ý: Bạn sẽ thay đổi tham số này để làm bài tập).*

```
class MLP(nn.Module):
    def __init__(self, input_dims, hidden_dims, output_dims):
        super(MLP, self).__init__()
        self.layer1 = nn.Linear(input_dims, hidden_dims)
        self.layer2 = nn.Linear(hidden_dims, hidden_dims)
        self.layer3 = nn.Linear(hidden_dims, hidden_dims)
        self.layer4 = nn.Linear(hidden_dims, hidden_dims)
        self.layer5 = nn.Linear(hidden_dims, hidden_dims)
        self.output = nn.Linear(hidden_dims, output_dims)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        x = nn.Flatten()(x)
        x = self.layer1(x)
        x = self.sigmoid(x)
        x = self.layer2(x)
        x = self.sigmoid(x)
        x = self.layer3(x)
        x = self.sigmoid(x)
        x = self.layer4(x)
        x = self.sigmoid(x)
        x = self.layer5(x)
        x = self.sigmoid(x)
        out = self.output(x)
        return out
```

3. Huấn luyện và Đánh giá (Train và Evaluate)

Quá trình huấn luyện được thực hiện qua vòng lặp `num_epochs`:

- **Giai đoạn Huấn luyện (Train):**

1. Chuyển mô hình sang chế độ `model.train()`.
2. Đặt Gradient về 0 (`zero_grad()`) để tránh tích lũy từ các batch trước.
3. Thực hiện Forward pass để tính toán dự đoán và giá trị Loss.
4. Lan truyền ngược (`backward()`) để tính toán Gradient.
5. Cập nhật trọng số thông qua phương thức `step()` của Optimizer.

- **Giai đoạn Đánh giá (Evaluate):**
  1. Chuyển mô hình sang chế độ `model.eval()`.
  2. Đưa tập test qua mô hình (không tính đạo hàm) để lấy giá trị Loss và Accuracy kiểm thử.
- **Lưu trữ và Ghi chép:** Toàn bộ thông số *train_loss, train_acc, val_loss, val_acc* được lưu vào list để phục vụ việc vẽ đồ thị so sánh giữa các Optimizer.

```
model = MLP(input_dims=784, hidden_dims=128, output_dims=10).to(device)
criterion = nn.CrossEntropyLoss()
################## YOUR CODE HERE ##################
"""Cấu hình optimizer theo yêu cầu đề bài"""
optimizer =
####################################################
```

```
train_losses = []
train_acc = []
val_losses = []
val_acc = []
for epoch in range(num_epochs):
    model.train()
    t_loss = 0
    t_acc = 0
    cnt = 0
    for X, y in train_loader:
        X, y = X.to(device), y.to(device)
        optimizer.zero_grad()
        outputs = model(X)
        loss = criterion(outputs, y)
        loss.backward()
        optimizer.step()
        t_loss += loss.item()
        t_acc += (torch.argmax(outputs, 1) == y).sum().item()
        cnt += len(y)
    t_loss /= len(train_loader)
    train_losses.append(t_loss)
    t_acc /= cnt
    train_acc.append(t_acc)

    model.eval()
    v_loss = 0
    v_acc = 0
```

```
    cnt = 0
    with torch.no_grad():
        for X, y in test_loader:
            X, y = X.to(device), y.to(device)
            outputs = model(X)
            loss = criterion(outputs, y)
            v_loss += loss.item()
            v_acc += (torch.argmax(outputs, 1)==y).sum().item()
            cnt += len(y)
    v_loss /= len(test_loader)
    val_losses.append(v_loss)
    v_acc /= cnt
    val_acc.append(v_acc)
    print(f"Epoch {epoch+1}/{num_epochs}, Train_Loss: {t_loss:.4f}, Train_Acc: {t_acc:.
                                            4f}, Validation Loss: {v_loss:.4f}, Val_Acc
                                            : {v_acc:.4f}")
```

2. Yêu Cầu Bài Tập

Dựa trên hàm số mục tiêu đã được thiết lập ở phần trước:

$$f(w_1, w_2) = 0.1w_1^2 + 2w_2^2$$

Các bạn thực hiện các yêu cầu cụ thể cho từng thuật toán tối ưu hóa dưới đây.

2.1 Thuật toán Gradient Descent

Sử dụng thuật toán Gradient Descent để tìm điểm cực tiểu của hàm số (1) với các tham số khởi tạo sau:

- **Tham số:** $w_1 = -5, \quad w_2 = -2$
- **Tốc độ học:** $\alpha = 0.4$

Nhiệm vụ:

a) **Tính toán thủ công:** Trình bày chi tiết từng bước thực hiện cập nhật để tìm giá trị w_1 và w_2 sau 2 epoch. (Yêu cầu trình bày bằng LaTeX hoặc file Doc).

b) **Lập trình NumPy:** Hiện thực mã nguồn bằng NumPy để tìm điểm cực tiểu sau 30 epoch.

2.2 Thuật toán Gradient Descent + Momentum

Sử dụng thuật toán Momentum để tìm điểm cực tiểu của hàm số (1) với các tham số khởi tạo sau:

- **Tham số:** $w_1 = -5, \quad w_2 = -2$
- **Vận tốc ban đầu:** $v_1 = 0, \quad v_2 = 0$
- **Hyperparameters:** $\alpha = 0.6, \quad \beta = 0.5$

Nhiệm vụ:

a) **Tính toán thủ công:** Trình bày chi tiết các bước cập nhật vận tốc và vị trí để tìm w_1, w_2 sau 2 epoch.

b) **Lập trình NumPy:** Hiện thực mã nguồn bằng NumPy để tìm điểm cực tiểu sau 30 epoch.

2.3 Thuật toán RMSProp

Sử dụng thuật toán RMSProp để tìm điểm cực tiểu của hàm số (1) với các tham số khởi tạo sau:

- **Tham số:** $w_1 = -5, \quad w_2 = -2$
- **Trạng thái ban đầu:** $s_1 = 0, \quad s_2 = 0$
- **Hyperparameters:** $\alpha = 0.3, \quad \gamma = 0.9, \quad \epsilon = 10^{-6}$

Nhiệm vụ:

a) **Tính toán thủ công:** Trình bày chi tiết các bước cập nhật sau 2 epoch.

b) **Lập trình NumPy:** Hiện thực mã nguồn bằng NumPy để tìm điểm cực tiểu sau 30 epoch.

2.4 Thuật toán Adam

Sử dụng thuật toán Adam để tìm điểm cực tiểu của hàm số (1) với các tham số khởi tạo sau:

- **Tham số:** $w_1 = -5, \quad w_2 = -2$
- **Trạng thái ban đầu:** $v_1 = 0, v_2 = 0, \quad s_1 = 0, s_2 = 0$
- **Hyperparameters:** $\alpha = 0.2, \quad \beta_1 = 0.9, \quad \beta_2 = 0.999, \quad \epsilon = 10^{-6}$

Nhiệm vụ:

a) **Tính toán thủ công:** Trình bày chi tiết 2 epoch đầu tiên (bao gồm cả bước Bias Correction).

b) **Lập trình NumPy:** Hiện thực mã nguồn bằng NumPy để tìm điểm cực tiểu sau 30 epoch.

2.5 Thử nghiệm Vấn đề Vanishing Gradient (Optional)

Phần bài tập này yêu cầu bạn thay đổi các Optimizer khác nhau trên cùng một kiến trúc mạng MLP để quan sát khả năng giảm thiểu vấn đề *Vanishing Gradient*.

Thiết lập mô hình tiêu chuẩn:

- **Khởi tạo trọng số:** Normal Distribution ($\mu = 0, \sigma = 0.05$).
- **Hàm mất mát:** Cross Entropy.
- **Kiến trúc:** 5 Hidden Layers, mỗi layer 128 Nodes.
- **Hàm kích hoạt:** Sigmoid.

Nhiệm vụ: Chạy thực nghiệm mô hình trên với các thuật toán tối ưu hóa sau và so sánh kết quả:

1. Gradient Descent (Baseline)

2. Gradient Descent + Momentum
3. RMSProp
4. Adam
5. ADOPT (Thuật toán nâng cao)

Câu hỏi bài tập

1. Với bài tập **Gradient Descent (a)** tại **epoch = 2**, kết quả w_1 và w_2 thu được là:
   (a) $-4.232, -0.72$.
   (b) $-4.232, -0.83$.
   (c) $-4.232, -0.94$.
   (d) $-4.232, -0.75$.
2. Với bài tập **Gradient Descent (b)** tại **epoch cuối cùng**, kết quả w_1 và w_2 gần nhất là:
   (a) $-3.098e-01, -4.521e-07$.
   (b) $-3.098e-01, -4.421e-07$.
   (c) $-4.098e-01, -4.521e-07$.
   (d) $-4.098e-01, -4.421e-07$.
3. Với bài tập **Gradient Descent + Momentum (a)** tại **epoch = 2**, kết quả w_1 và w_2 là:
   (a) $-4.468, 1.22$.
   (b) $-4.368, 1.32$.
   (c) $-4.268, 1.12$.
   (d) $-4.568, 1.42$.
4. Với bài tập **Gradient Descent + Momentum (b)** tại **epoch cuối cùng**, kết quả w_1 và w_2 là:
   (a) $-7.1e-03, 6.45e-05$.
   (b) $-6.1e-03, 7.45e-06$.
   (c) $-7.1e-02, 7.45e-06$.
   (d) $-6.1e-02, 6.45e-05$.
5. Cho gradient 1 chiều g_t tại thời điểm t có giá trị 2 và sử dụng momentum với $\beta = 0.9$, giá trị moving average m_t sẽ là bao nhiêu nếu m_{t-1} là 1?
   (a) 1.1.
   (b) 1.9.

(c) 2.9.

(d) 0.01.

6. Với bài tập **RMSProp (a)** tại **epoch = 2**, kết quả w_1 và w_2 thu được là:

   (a) $-3.436, -0.791$.

   (b) $-4.436, -0.791$.

   (c) $-4.436, -0.591$.

   (d) $-3.436, -0.591$.

7. Với bài tập **RMSProp (b)** tại **epoch cuối cùng**, kết quả w_1 và w_2 là:

   (a) $-3.00577e-04, -3.005e-18$.

   (b) $-3.00577e-03, -3.005e-17$.

   (c) $-3.00577e-02, -3.005e-16$.

   (d) $-3.00577e-01, -3.005e-15$.

8. Với bài tập **Adam (a)** tại **epoch = 2**, kết quả w_1 và w_2 thu được là:

   (a) $-4.6002546, -1.6008245$.

   (b) $-5.6002546, -2.6008245$.

   (c) $-6.6002546, -3.6008245$.

   (d) $-7.6002546, -4.6008245$.

9. Với bài tập **Adam (b)** tại **epoch cuối cùng**, kết quả w_1 và w_2 là:

   (a) $-0.71, 0.0679$.

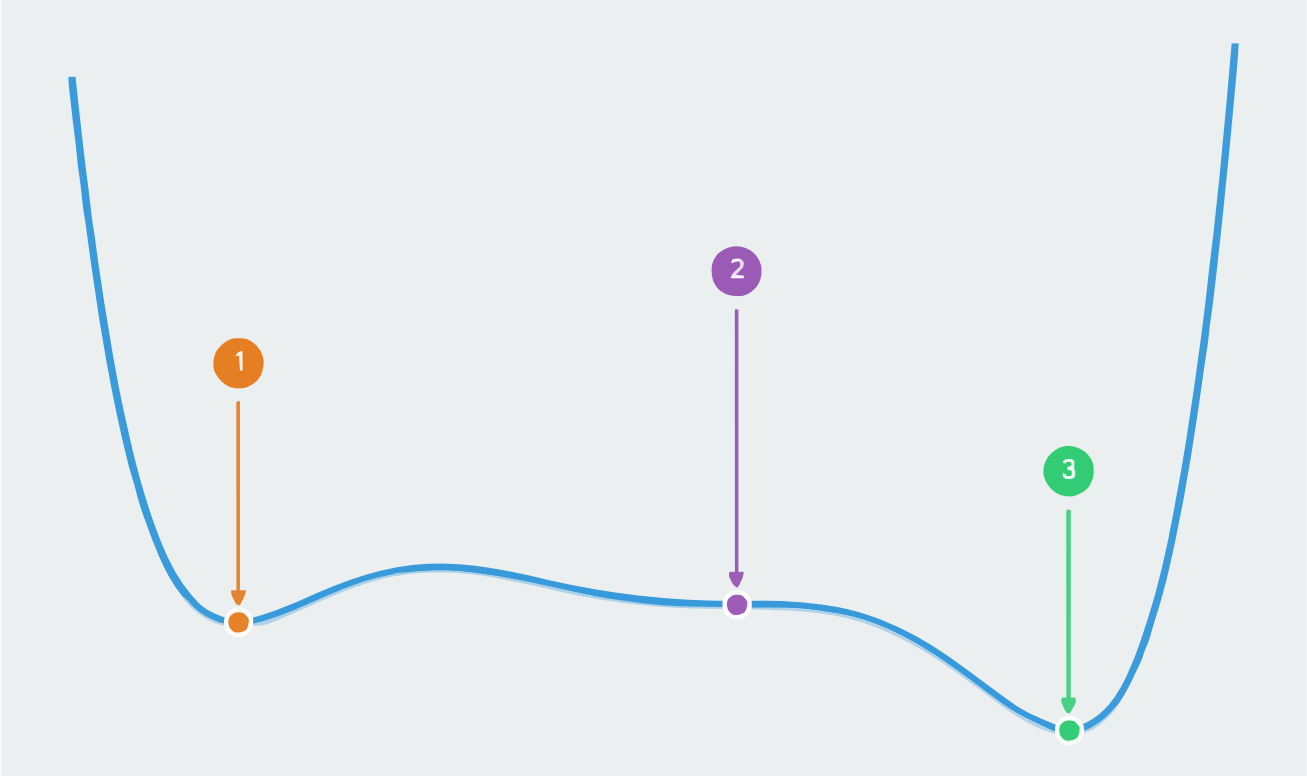
   (b) $-0.51, 0.0679$.

   (c) $-0.11, 0.0679$.

   (d) $-0.31, 0.0679$.

IV. Câu hỏi trắc nghiệm

1. Quan sát đồ thị biểu diễn hàm mất mát của một mô hình dưới đây. Hãy xác định tên gọi chính xác của các điểm dừng được đánh số 1, 2, và 3.



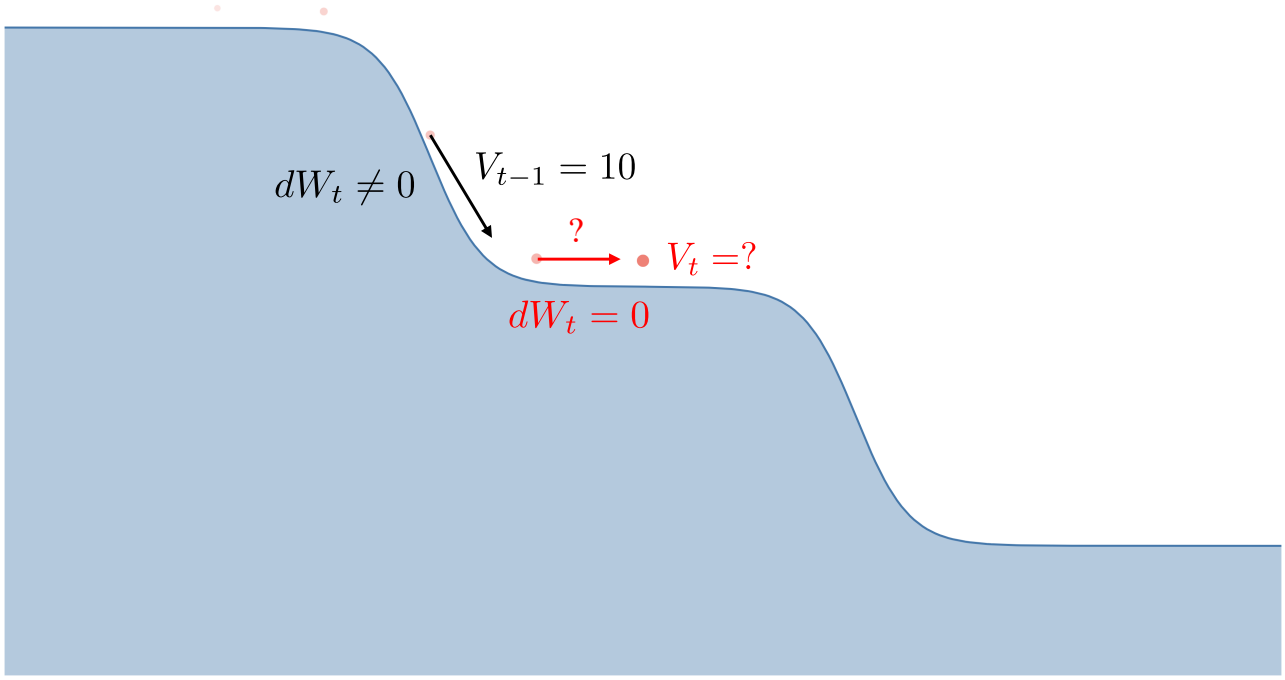
   (*a*) 1: Global Minimum, 2: Saddle Point, 3: Local Minimum.

   (*b*) 1: Local Minimum, 2: Saddle Point, 3: Global Minimum.

   (*c*) 1: Local Minimum, 2: Global Minimum, 3: Saddle Point.

   (*d*) 1: Saddle Point, 2: Local Maximum, 3: Global Minimum.

2. Xét thuật toán **Momentum** với công thức: $V_t = \beta V_{t-1} + (1-\beta)dW_t$. Giả sử tại bước t, viên bi lăn vào vùng phẳng có $dW_t = 0$. Nếu trước đó vận tốc tích lũy là $V_{t-1} = 10$ và hệ số quán tính $\beta = 0.9$, hãy tính giá trị bước nhảy cập nhật tại thời điểm này.



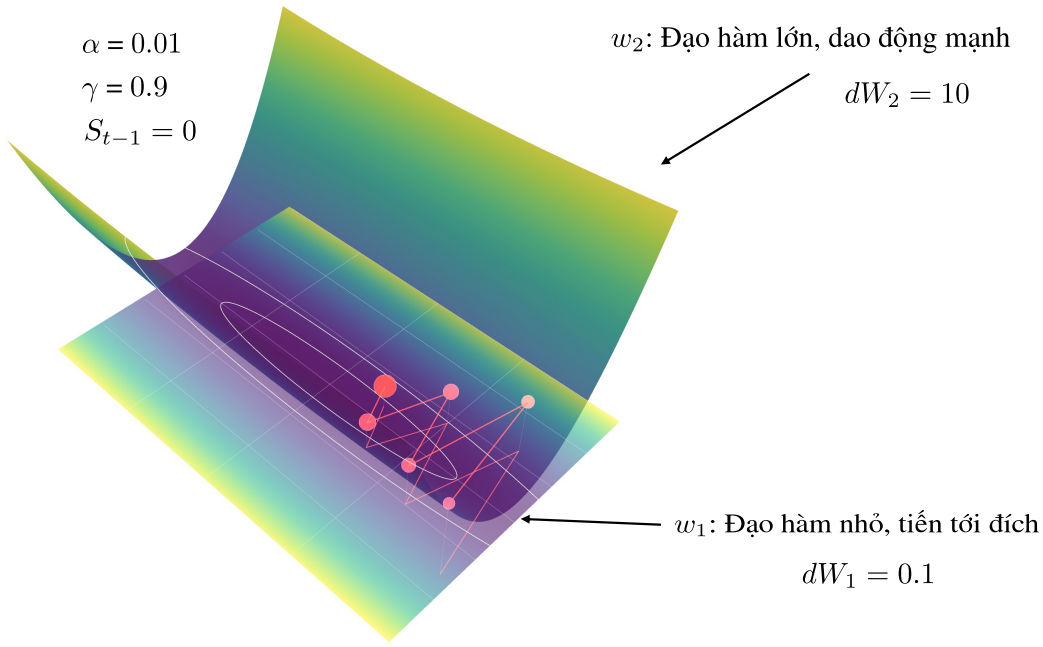
(*a*) 0 (Viên bi dừng lại ngay lập tức).

(*b*) 1

(*c*) 9.

(*d*) 10

3. Trong một narrow valley nơi đạo hàm theo trục tung (w_2) lớn hơn rất nhiều so với trục hoành (w_1), hình vẽ của **RMSProp** được thể hiện như sau. Bỏ qua tham số ϵ. Tham số cập nhật của ΔW_1 và ΔW_2 **chênh lệch** nhau bao nhiêu (lấy ΔW_1 trừ cho ΔW_2)?



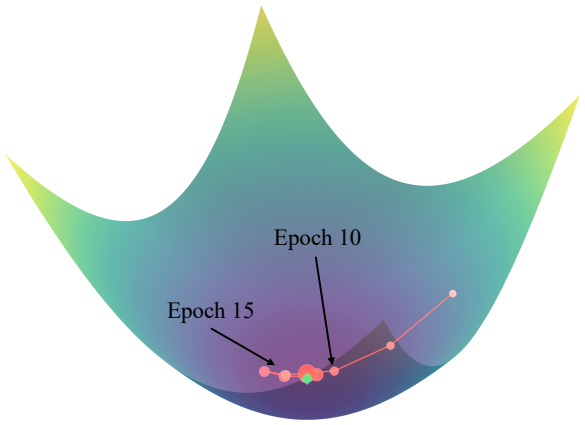
(a) -0.101

(b) 0.1

(c) 0.101

(d) 0

4.



Dựa trên hình ảnh minh họa về sự hội tụ, bộ tối ưu nào có xu hướng dễ xảy ra hiện tượng "vượt rào" (overshoot) – tức là đi quá điểm cực tiểu toàn cục do vận tốc tích lũy quá lớn?

(a) Gradient Descent truyền thống.

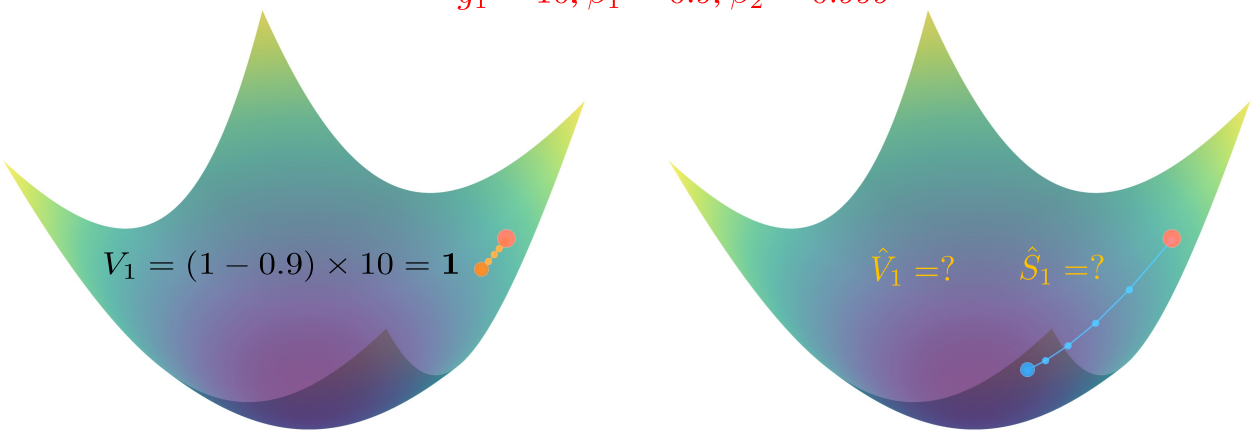
(b) Momentum và Adam.

(c) Chỉ duy nhất RMSProp.

(d) Cả 4 bộ tối ưu đều dừng lại ngay lập tức khi chạm đáy.

5. Giả sử tại bước khởi tạo $t = 1$, đạo hàm thu được là $g_1 = 10$. Với các tham số mặc định $\beta_1 = 0.9$ và $\beta_2 = 0.999$, hãy tính giá trị trung bình động $(\hat{V}_1, \hat{S}_1)$ sau khi thực hiện **Bias Correction** và so sánh với giá trị chưa hiệu chỉnh (V_1, S_1).

$g_1 = 10, \beta_1 = 0.9, \beta_2 = 0.999$



$S_1 = (1 - 0.999) \times 10^2 = 0.001 \times 100 = \mathbf{0.1}$

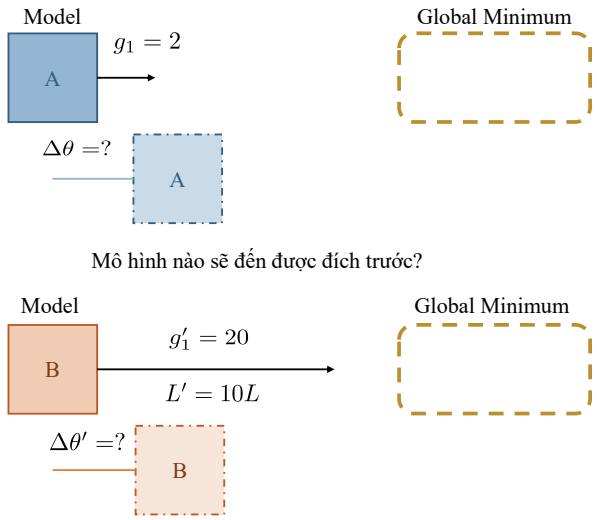
(a) $\hat{V}_1 = 1, \hat{S}_1 = 0.1$.

(b) $\hat{V}_1 = 10, \hat{S}_1 = 100$.

(c) $\hat{V}_1 = 10, \hat{S}_1 = 10$.

(d) $\hat{V}_1 = 1, \hat{S}_1 = 1$

6.



Ta huấn luyện một mô hình song song với hàm hàm Loss khác nhau. Giả sử tại bước $t = 1$, với hàm L, ta thu được đạo hàm $g_1 = 2$. Với hàm $L' = 10L$, đạo hàm thu được là $g_1' = 20$.

Cho các tham số của Adam như sau: $\alpha = 0.01, \beta_1 = 0.9, \beta_2 = 0.999$ và bỏ qua ϵ. Hãy tính tỉ số giữa bước cập nhật của hàm L' và hàm L (tức là $\frac{\Delta\theta'}{\Delta\theta}$).

(a) 10.

(b) 0.1.

(c) 1.

(d) 100.

7. Để ổn định quá trình huấn luyện với Adam, trong những bước đầu chúng ta sử dụng **Linear Warmup** với 100 bước khởi động. Nếu tốc độ học mục tiêu (target) là $\alpha = 0.01$, hãy tính tốc độ học thực tế tại bước thứ $t = 30$ theo công thức $\alpha_t = \frac{t}{T_{warmup}} \times \alpha$.

   (a) 0.001

   (b) 0.003

   (c) 0.01

   (d) 0.3

8. Trong quá trình tối ưu, các trọng số W có thể tăng lên rất lớn dẫn đến hiện tượng Overfitting. Kỹ thuật Weight Decay (thường dùng trong AdamW hoặc GD) sẽ chủ động "phạt" và kéo trọng số nhỏ lại sau mỗi bước cập nhật để mô hình bền vững hơn.

   Khi sử dụng bộ tối ưu kèm theo **Weight Decay** (λ), trọng số W sẽ được cập nhật nhẹ trước khi cộng gradient: $W_{new} = W - \eta(\lambda W)$. Với trọng số hiện tại $W = 10$, tốc độ học $\eta = 0.1$ và hệ số phạt $\lambda = 0.05$, giá trị trọng số sau khi bị "phạt" (decay) là bao nhiêu?

   (a) 9.5

   (b) 9.95

   (c) 10.05

   (d) 9.9

9. Phương pháp **Newton** cập nhật trọng số theo công thức $w_{new} = w_{old} - \frac{f'(w)}{f''(w)}$. Xét hàm Loss $f(w) = w^2$. Tại điểm $w = 4$, ta có đạo hàm bậc một $f'(4) = 8$ và đạo hàm bậc hai $f''(4) = 2$. Sau **một bước** cập nhật duy nhất bằng phương pháp Newton, giá trị w mới sẽ là:

   (a) 0

   (b) 2

   (c) -4

   (d) 4

10. Xét một mô hình mạng Neural nhỏ có $n = 1.000$ tham số. Để thực hiện một bước cập nhật:

    - Gradient Descent (GD) cần tính toán n đạo hàm bậc một.
    - Phương pháp Newton cần tính toán ma trận Hessian chứa n^2 đạo hàm bậc hai.

    Hãy tính xem số lượng phần tử cần tính toán của phương pháp Newton gấp bao nhiêu lần so với GD trong trường hợp này?

    (a) 10 lần

    (b) 100 lần

    (c) 1.000 lần

    (d) 1.000.000 lần

V. Tài liệu tham khảo
Phụ lục

1. **Hint:** Các file code gợi ý có thể tải tại Link
2. **Solution:** Các file code cài đặt hoàn chỉnh và phần trả lời nội dung trắc nghiệm có thể tải tại Link (Lưu ý Sáng thứ 3 khi hết deadline phần bài tập ad mới copy các nội dung bài giải nêu trên vào đường dẫn)
3. **Q&A:** Bạn có thể đặt thêm câu hỏi về nội dung bài đọc trong group Facebook hỏi đáp tại đây. Tất cả câu hỏi sẽ được trả lời trong vòng 3-4 tiếng.



Hình 12: Hình ảnh group facebook AIO Q&A.